

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Reminder: Homework Assignment 1

- Due: **11:59 PM** on **Wednesday, February 11**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
 - Do not upload the data file.
 - Make sure to **rename your notebook file** as instructed.

Lecture 6

- Data Wrangling
 - Groupby and Split-Apply-Combine

Overview

- In this session, we will tackle the handy groupby method.
- We also cover the split-apply-combine paradigm to:
 - Aggregate data in each group
 - Transform data in each group
 - Filter the data in each group
 - Thin the data in each group

```

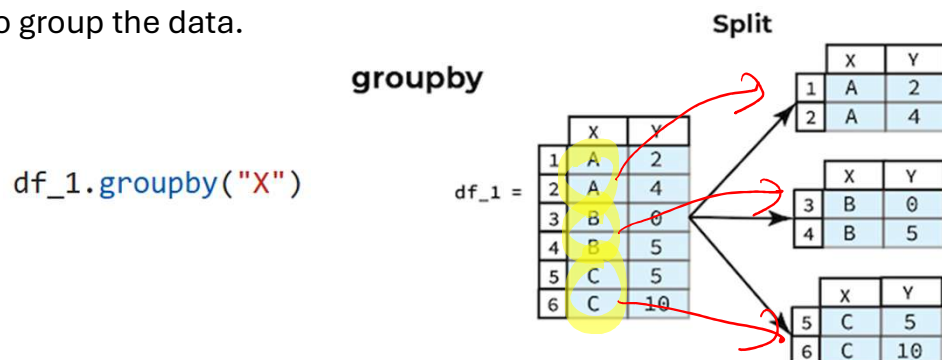
spending_df = pd.read_csv('data/spending_10k.csv', index_col="unique_id", dtype={"doctor_id": "object"})
spending_df.head(10)

```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
NX531425	1255626040	FAMILY PRACTICE	METFORMIN HCL	30	135.24
QG879256	1699761833	FAMILY PRACTICE	ALLOPURINOL	30	715.76
FW363228	1538148804	INTERNAL MEDICINE	LOSARTAN POTASSIUM	146	1056.47
WD733417	1730200619	PSYCHIATRY	OLANZAPINE	13	28226.97
XW149832	1023116894	FAMILY PRACTICE	PRAVASTATIN SODIUM	348	8199.48
QT485324	1952359671	FAMILY PRACTICE	HYDROCHLOROTHIAZIDE	57	247.01
NA293426	1841235223	FAMILY PRACTICE	SEVELAMER CARBONATE	11	4869.32
IF945618	1326095662	INTERNAL MEDICINE	FLUTICASONE/SALMETEROL	20	7832.46
PH384257	1821126830	HEMATOLOGY/ONCOLOGY	ZOLPIDEM TARTRATE	14	65.21
JY407340	1710986088	INTERNAL MEDICINE	MECLIZINE HCL	47	861.67

groupby and DataFrame groups

- The `groupby()` method is used to group the data using values from one or more columns.
- `groupby()` takes as input one or more column labels, which it uses to group the data.



Identifying groups from a GroupBy object

- The `groupby()` method returns an object of type `DataFrameGroupBy`.
 - This is not a `DataFrame` and does not, therefore, have the `DataFrame` methods discussed previously.
- We can get `DataFrameGroupBy` element using a valid `groupby` column(s) value(s).

`spending_df.groupby('specialty')`

	specialty	Spending
VG585760	ADDICTION MEDICINE	82.62
PI452198	GERIATRIC PSYCHIATRY	2280.71
WA740528	GERIATRIC PSYCHIATRY	1493.43
GJ278932	ADDICTION MEDICINE	817.88
FU325338	GERIATRIC PSYCHIATRY	9318.09
TX420809	ADDICTION MEDICINE	19.56
CW959540	GERIATRIC PSYCHIATRY	4176.32

	specialty	Spending
VG585760	ADDICTION MEDICINE	82.62
GJ278932	ADDICTION MEDICINE	817.88
TX420809	ADDICTION MEDICINE	19.56

	specialty	Spending
PI452198	GERIATRIC PSYCHIATRY	2280.71
WA740528	GERIATRIC PSYCHIATRY	1493.43
FU325338	GERIATRIC PSYCHIATRY	9318.09
CW959540	GERIATRIC PSYCHIATRY	4176.32

```
x = spending_df.groupby('specialty')
type(x)
```

`pandas.core.groupby.generic.DataFrameGroupBy`

```
spending_by_specialty = spending_df.groupby('specialty')
```

```
addiction_med_group = spending_by_specialty.get_group("ADDICTION MEDICINE")
addiction_med_group
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
VG585760	1801032297	ADDICTION MEDICINE	LAMOTRIGINE	11	82.62
GJ278932	1134139991	ADDICTION MEDICINE	BUSPIRONE HCL	49	817.88
TX420809	1801032297	ADDICTION MEDICINE	LORAZEPAM	14	19.56

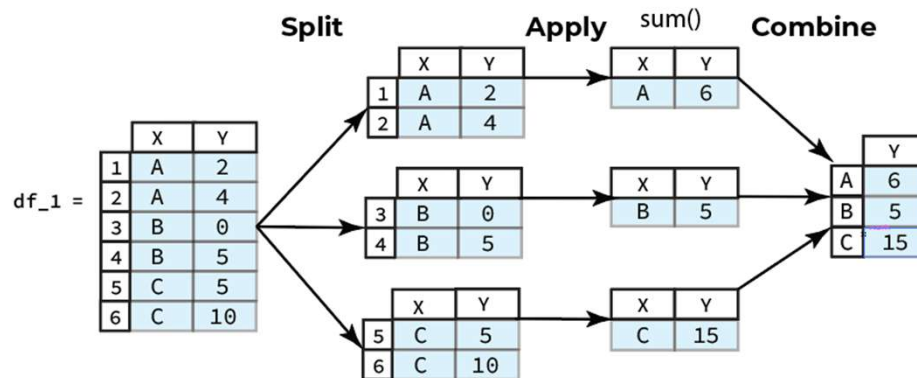
groupby and group-specific processing

- An ideal use-case for `groupby` consists of applying operations to each group independently.
- For instance, to compute the total spending by `specialty`, we need to:
 - Split the data by `specialty`.
 - Sum the total spending for each type of `specialty`.
 - Combine and return the sums for each group into as a new `DataFrame`.

Split-Apply-Combine paradigm

- `groupby()` is often applied in the context of the data processing paradigm called “split-apply-combine”.
 - **Split:** You need to split the data into chunks defined using one or more columns.
 - This is typically done using `groupby()`.
 - **Apply:** Apply some operation to the chunks generated.
 - Ex. Count the number of rows in each chunk, average the values for a specific column, etc.
 - **Combine:** Combine the results of the applied operation into a new `DataFrame`.

Split-Apply-Combine example



- The type of split-apply-combine applied here is referred to as *aggregation*.
 - Aggregations refer to any operation that aggregates (reduces) group data to a single value.

The 3 (or $3\frac{1}{2}$) classes of operations on groups

- There are three formal classes of split-apply-combine operations that can be applied to group data.
 - I include a variant ($\frac{1}{2}$ a class) which I think is useful to better classify split-apply-combine operations.
- 1. Aggregations** generate a single value for each group.
 - E.g., sum the spending by specialty.

The 3 (or $3\frac{1}{2}$) classes of operations on groups

2. Transformations convert the data and generate a group of the same size as the original group.

- E.g., convert the currency by country for some datasets that contains medication cost by country.

3. Filtering retains or discards a group based on group-specific Boolean computations.

- E.g., drop specialty if the sum of spending is below some threshold.

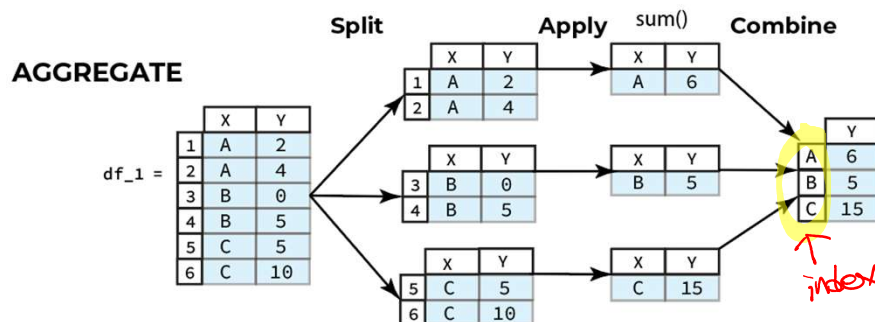
$3\frac{1}{2}$. Thinning drops entries in a group based on some defined logic.

- E.g., retain only those group entries whose values lie within a specified distance from the group mean.

Aggregations

• **Aggregations** aggregate the data in each group, i.e., they **reduce** the data in each group to **a single value**.

- This includes, for instance, computing group sums, means, maximums, minimums, etc.

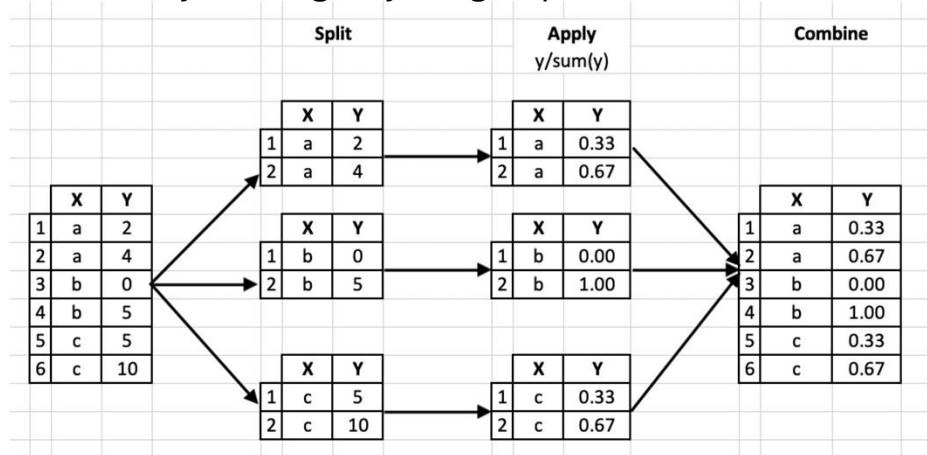


Transforming group data

- **Transformations** will transform the data in a group-specific way.
 - E.g., transform the value by dividing it by the group sum.
- The number of entries per group resulting from a transformation is the same as the number of entries in the group before the transformation.

Transforming group data (cont'd.)

- The figure below shows an example where the data in column Y is transformed by dividing it by the group sum.

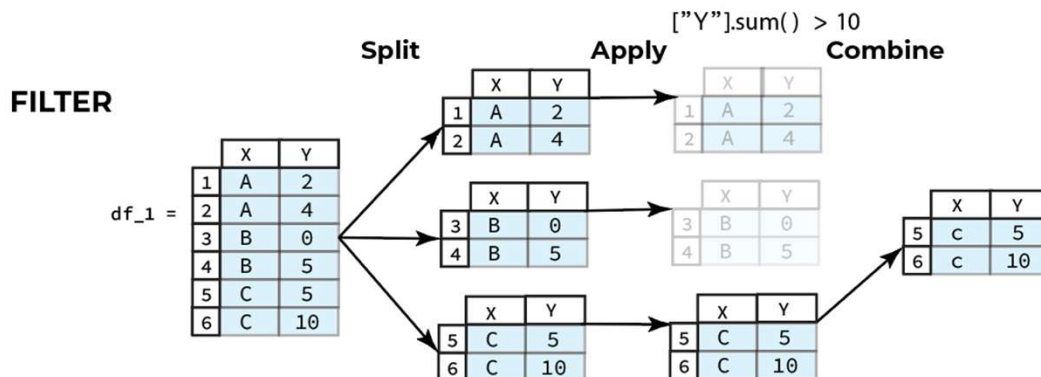


Filtering group data

- Consist of dropping or retaining a group in a way that depends on a group-specific computation that returns True or False.
- For instance, we can filter specialties that don't have enough entries or for which the mean spending is below a certain threshold.
 - Groups are either retained or discarded. Groups that are retained are unmodified.

Filtering group data (cont'd.)

- The figure below shows an example where groups are filtered if their sum for column Y is less than 10.

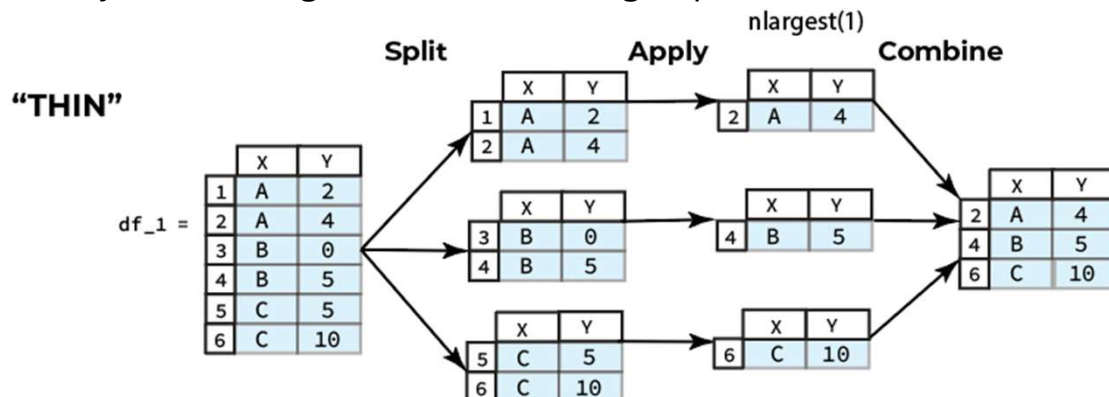


Thinning group data

- Consist of reducing the number of entries using a group-specific operation.
- Thinning can be useful to, for instance:
 - Sub-sample the data at the group level (create a balanced dataset),
 - Returning the top n entries in each group, etc.
- As opposed to aggregating functions, thinning does not have to reduce the group into a single entry; although it could.

Thinning group data (cont'd.)

- The figure below illustrates a thinning operation where only the entry with the largest Y value in each group is retained..



df.size

Aggregating the data using groupby

- Aggregation is commonly used to compute summary statistics on each of the groups.
- Some of the interesting/important summary aggregation methods DataFrameGroupBy objects have:

Methods	Description
mean , median	Computes the mean or the median for each group
min , max	Computes the minimum or maximum for each group
size	Counts the number of values in each group

Groupby.size()

Aggregating the data using groupby (cont'd.)

- The function applies to all valid columns that are available after using the groupby argument as an index.
- The functions in the previous slide all use the same syntax:

```
spending_df.groupby('specialty').sum()  
# or  
spending_df.groupby('specialty').min()
```

```
df = pd.DataFrame({"A": [1, 1, 2, 2, 3, 3], "B": [1, 2, 3, 4, 5, 6],
                  "C": ["A", "B", "C", "D", "E", "F"]})
display(df)

df.groupby("A").sum()
```

	A	B	C
0	1	1	A
1	1	2	B
2	2	3	C
3	2	4	D
4	3	5	E
5	3	6	F

	A	B	C
1	3	AB	
2	7	CD	
3	11	EF	

```
spending_df.head()
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
NX531425	1255626040	FAMILY PRACTICE	METFORMIN HCL	30	135.24
QG879256	1699761833	FAMILY PRACTICE	ALLOPURINOL	30	715.76
FW363228	1538148804	INTERNAL MEDICINE	LOSARTAN POTASSIUM	146	1056.47
WD733417	1730200619	PSYCHIATRY	OLANZAPINE	13	28226.97
XW149832	1023116894	FAMILY PRACTICE	PRAVASTATIN SODIUM	348	8199.48

```
spending_df.groupby('specialty').sum().head(10)
```

specialty	doctor_id	medication	nb_beneficiaries	spending
ADDICTION MEDICINE	180103229711341399911801032297	LAMOTRIGINEBUSPIRONE HCLLORAZEPAM	74	920.06
ALLERGY/IMMUNOLOGY	1104888403100389785118614997411720080062179085...	DEXLANSOPRAZOLEMOMETASONE FUROATEALBUTEROL SUL...	1063	189174.06
ANESTHESIOLOGY	1972509974140783819618918500121760454805118461...	HYDROCODONE/IBUPROFENATENOLOLHYDROMORPHONE HCL...	1673	142804.73
CARDIAC ELECTROPHYSIOLOGY	1710150693103321883918210567711538130034166958...	RIVAROXABANLISINOPRILRIVAROXABANFLECAINIDE ACE...	1041	225543.62
CARDIAC SURGERY	16896272671609804855	INSULIN GLARGINE,HUM.REC.ANLOGHYDROCODONE/ACET...	33	12432.92
CARDIOLOGY	1013915552175038231316191814271801882386156846...	ESOMEPRAZOLE MAGNESIUMPOTASSIUM CHLORIDEPRAS...	29638	1915787.90
CERTIFIED CLINICAL NURSE SPECIALIST	1528385762152838576216897420741437306941182115...	METOPROLOL SUCCINATEMETFORMIN HCLROPINIROLE HC...	1146	114057.40
CERTIFIED NURSE MIDWIFE	16290689601942202890	MIRABEGRONMEDROXYPROGESTERONE ACETATE	58	14763.47
CLINIC/CENTER	17606802431730415829	ESOMEPRAZOLE MAGNESIUMMIRTAPAZINE	23	3110.16
CLINICAL PSYCHOLOGIST	1235211798	ZOLPIDEM TARTRATE	83	495.95

Applying functions to group columns

- The method `agg()` can be used where complex or custom aggregation logic is required.
- In this case, the `agg()` method applies to a `DataFrameGroupBy` object.
 - The method `agg()` takes a function (or a list of functions) and uses it (them) to aggregate the group's column(s).
- For example, we can use `sum_spending_CAD` to return the sum of the spending in Canadian dollars.

`def sum_spending_CAD(x):`
 `return x.sum() * 1.39`

`spending_by_specialty['spending'].agg(sum_spending_CAD)`

`specialty`
`ADDICTION MEDICINE`
`ALLERGY/IMMUNOLOGY`
`ANESTHESIOLOGY`
`CARDIAC ELECTROPHYSIOLOGY`
`CARDIAC SURGERY`

`STUDENT IN AN ORGANIZED HEALTH CARE EDUCATION/TRAINING PROGRAM`
`THORACIC SURGERY`
`UNKNOWN PHYSICIAN SPECIALTY CODE`
`UROLOGY`
`VASCULAR SURGERY`
`Name: spending, Length: 75, dtype: float64`

1278.8834
 262951.9434
 198498.5747
 313505.6318
 17281.7588
 ...
 29617.0775
 261.5980
 5744.4391
 490752.7884
 6070.3802

group ['spending']
 total spending in CAD
 GroupBy obj.

Applying functions to group columns (cont'd.)

- `agg()` can take a dictionary of functions to aggregate on.
 - Required for aggregating more than one column.

```
spending_by_specialty.agg({'nb_beneficiaries': "sum",
                           'spending': sum_spending_CAD}).head()
```

	nb_beneficiaries	spending
specialty		
ADDICTION MEDICINE	74	1278.8834
ALLERGY/IMMUNOLOGY	1063	262951.9434
ANESTHESIOLOGY	1673	198498.5747
CARDIAC ELECTROPHYSIOLOGY	1041	313505.6318
CARDIAC SURGERY	33	17281.7588

Applying functions to group columns (cont'd.)

- `agg()` can take a list of functions to apply to each column to aggregate on.

```
# note that sum, min and max here are functions
spending_by_specialty[['spending', 'nb_beneficiaries']].agg(["sum", "min", "max"]).head()
```

specialty	spending			nb_beneficiaries		
	sum	min	max	sum	min	max
ADDICTION MEDICINE	920.06	19.56	817.88	74	11	49
ALLERGY/IMMUNOLOGY	189174.06	109.80	52389.61	1063	11	297
ANESTHESIOLOGY	142804.73	35.33	34073.91	1673	12	419
CARDIAC ELECTROPHYSIOLOGY	225543.62	69.85	89101.54	1041	12	188
CARDIAC SURGERY	12432.92	442.91	11990.01	33	15	18

```
# same operation as above
func_list = ["sum", "min", "max"]

spending_by_specialty.agg({'spending': func_list,
                           'nb_beneficiaries': func_list}).head()
```

specialty	spending			nb_beneficiaries		
	sum	min	max	sum	min	max
ADDICTION MEDICINE	920.06	19.56	817.88	74	11	49
ALLERGY/IMMUNOLOGY	189174.06	109.80	52389.61	1063	11	297
ANESTHESIOLOGY	142804.73	35.33	34073.91	1673	12	419
CARDIAC ELECTROPHYSIOLOGY	225543.62	69.85	89101.54	1041	12	188
CARDIAC SURGERY	12432.92	442.91	11990.01	33	15	18

```

spending_by_specialty.agg({'nb_beneficiaries': ["min", "sum"],
                           'spending': "max"}).head()

```

	nb_beneficiaries		spending
	min	sum	max
specialty			
ADDICTION MEDICINE	11	74	817.88
ALLERGY/IMMUNOLOGY	11	1063	52389.61
ANESTHESIOLOGY	12	1673	34073.91
CARDIAC ELECTROPHYSIOLOGY	12	1041	89101.54
CARDIAC SURGERY	15	33	11990.01

Transforming the data in groupby

- As opposed to aggregations, which reduce the data into a single value, transformations modify the data but don't change the shape (dimension) of the groups.
- Transformations are useful for applying operations that are group specific.

Transforming the data in groupby (cont'd.)

- The example below computes the percent contribution of each entry to its specialty by applying a transformation that normalizes the entry's spending over the total spending in that specialty.

	specialty	Spending
VG585760	ADDICTION MEDICINE	82.62
PI452198	GERIATRIC PSYCHIATRY	2280.71
WA740528	GERIATRIC PSYCHIATRY	1493.43
GJ278932	ADDICTION MEDICINE	817.88
FU325338	GERIATRIC PSYCHIATRY	9318.09
TX420809	ADDICTION MEDICINE	19.56
CW959540	GERIATRIC PSYCHIATRY	4176.32

specialty	Spending
ADDICTION MEDICINE	82.62
ADDICTION MEDICINE	817.88
ADDICTION MEDICINE	19.56
GERIATRIC PSYCHIATRY	2280.71
GERIATRIC PSYCHIATRY	1493.43
GERIATRIC PSYCHIATRY	9318.09
GERIATRIC PSYCHIATRY	4176.32

specialty	Spending
ADDICTION MEDICINE	82.62 / 920.06
ADDICTION MEDICINE	817.88 / 920.06
ADDICTION MEDICINE	19.56 / 920.06
GERIATRIC PSYCHIATRY	2280.71 / 17,268.55
GERIATRIC PSYCHIATRY	1493.43 / 17,268.55
GERIATRIC PSYCHIATRY	9318.09 / 17,268.55
GERIATRIC PSYCHIATRY	4176.32 / 17,268.55

specialty	Spending
ADDICTION MEDICINE	8.97
GERIATRIC PSYCHIATRY	13.2
GERIATRIC PSYCHIATRY	8.6
ADDICTION MEDICINE	88.89
GERIATRIC PSYCHIATRY	53
ADDICTION MEDICINE	2.12
GERIATRIC PSYCHIATRY	24.1

Applying a transformation

- Applying a transformation is done using the method called `transform()`.
- The method `transform()` takes as input a function name, which it calls on each group of the `DataFrameGroupBy` object.

```
def my_function(x):
    return (x / x.sum() ) * 100

spending_by_specialty['spending'].transform(my_function).head()
```

```
unique_id
NX531425    0.002075
QG879256    0.010985
FW363228    0.010855
WD733417    1.614442
XW149832    0.125835
Name: spending, dtype: float64
```

```
spending_df["spending_pct"] = spending_by_specialty['spending'].transform(my_function)
```

```
spending_df.head()
```

group by obj

	doctor_id	specialty	medication	nb_beneficiaries	spending	spending_pct
unique_id						
NX531425	1255626040	FAMILY PRACTICE	METFORMIN HCL	30	135.24	0.002075
QG879256	1699761833	FAMILY PRACTICE	ALLOPURINOL	30	715.76	0.010985
FW363228	1538148804	INTERNAL MEDICINE	LOSARTAN POTASSIUM	146	1056.47	0.010855
WD733417	1730200619	PSYCHIATRY	OLANZAPINE	13	28226.97	1.614442
XW149832	1023116894	FAMILY PRACTICE	PRAVASTATIN SODIUM	348	8199.48	0.125835

```
spending_df[spending_df['specialty'] == "ADDICTION MEDICINE"]
```

	doctor_id	specialty	medication	nb_beneficiaries	spending	spending_pct
unique_id						
VG585760	1801032297	ADDICTION MEDICINE	LAMOTRIGINE	11	82.62	8.979849
GJ278932	1134139991	ADDICTION MEDICINE	BUSPIRONE HCL	49	817.88	88.894203
TX420809	1801032297	ADDICTION MEDICINE	LORAZEPAM	14	19.56	2.125948

```
spending_df.sort_values(['specialty', 'spending_pct'], ascending=[True, False]).head(10)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending	spending_pct
unique_id						
GJ278932	1134139991	ADDICTION MEDICINE	BUSPIRONE HCL	49	817.88	88.894203
VG585760	1801032297	ADDICTION MEDICINE	LAMOTRIGINE	11	82.62	8.979849
TX420809	1801032297	ADDICTION MEDICINE	LORAZEPAM	14	19.56	2.125948
XY715196	1376691626	ALLERGY/IMMUNOLOGY	FLUTICASONE/SALMETEROL	102	52389.61	27.693866
DL492570	1962588053	ALLERGY/IMMUNOLOGY	OMALIZUMAB	12	29153.71	15.411051
UJ888112	1003897851	ALLERGY/IMMUNOLOGY	MOMETASONE FUROATE	55	20759.04	10.973513
GO641321	1255301404	ALLERGY/IMMUNOLOGY	FLUTICASONE/SALMETEROL	35	14277.61	7.547340
JU235992	1003812595	ALLERGY/IMMUNOLOGY	MOMETASONE FUROATE	50	13559.50	7.167737
WE196352	1720080062	ALLERGY/IMMUNOLOGY	FLUTICASONE/SALMETEROL	37	12594.63	6.657694
EW891894	1104888403	ALLERGY/IMMUNOLOGY	DEXLANSOPRAZOLE	32	12411.92	6.561111

More complex transformations

- As you may have noted in the previous slide, drugs are still duplicated across doctor_ids within the same specialty.
 - Ex., FLUTICASONE/SALMETEROL is prescribed by at least three doctors whose specialty is ALLERGY/IMMUNOLOGY.
- To see the percent spending by drug column, we need to group on both the specialty and the medication and then sum the spending_pct computed previously.

```
medication_spending_pct = (spending_df
    .groupby(["specialty", "medication"])["spending_pct"]
    .sum())
```

```
medication_spending_pct = (spending_df
    .groupby(["specialty", "medication"])["spending_pct"]
    .sum())
```

MultiIndex

```
medication_spending_pct.head(15)
```

specialty	medication	
ADDICTION MEDICINE	BUSPIRONE HCL	88.894203
	LAMOTRIGINE	8.979849
	LORAZEPAM	2.125948
ALLERGY/IMMUNOLOGY	ALBUTEROL SULFATE	3.553199
	AMOXICILLIN/POTASSIUM CLAV	0.196089
	AZELASTINE HCL	3.646451
	AZITHROMYCIN	0.100500
	DEXLANSOPRAZOLE	6.561111
	DILTIAZEM HCL	0.293666
	DOXAZOSIN MESYLATE	0.104872
	ENALAPRIL MALEATE	0.318553
	FLUTICASONE PROPIONATE	2.842509
	FLUTICASONE/SALMETEROL	41.898900
	HYDROXYZINE HCL	0.135473
	IRBESARTAN	0.058042

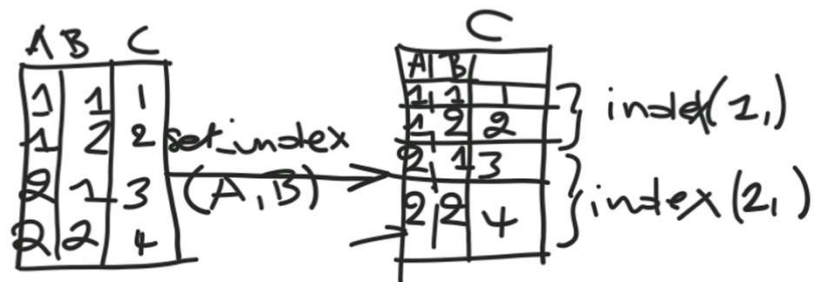
Name: spending_pct, dtype: float64

```
medication_spending_pct.index
```

```
MultiIndex([( 'ADDICTION MEDICINE', 'BUSPIRONE HCL'),
            ( 'ADDICTION MEDICINE', 'LAMOTRIGINE'),
            ( 'ADDICTION MEDICINE', 'LORAZEPAM'),
            ( 'ALLERGY/IMMUNOLOGY', 'ALBUTEROL SULFATE'),
            ( 'ALLERGY/IMMUNOLOGY', 'AMOXICILLIN/POTASSIUM CLAV'),
            ( 'ALLERGY/IMMUNOLOGY', 'AZELASTINE HCL'),
            ( 'ALLERGY/IMMUNOLOGY', 'AZITHROMYCIN'),
            ( 'ALLERGY/IMMUNOLOGY', 'DEXLANSOPRAZOLE'),
            ( 'ALLERGY/IMMUNOLOGY', 'DILTIAZEM HCL'),
            ( 'ALLERGY/IMMUNOLOGY', 'DOXAZOSIN MESYLATE'),
            ...
            ( 'UROLOGY', 'SILDENAFIL CITRATE'),
            ( 'UROLOGY', 'SULFAMETHOXAZOLE/TRIMETHOPRIM'),
            ( 'UROLOGY', 'TADALAFIL'),
            ( 'UROLOGY', 'TAMSULOSIN HCL'),
            ( 'UROLOGY', 'TESTOSTERONE'),
            ( 'UROLOGY', 'TOLTERODINE TARTRATE'),
            ( 'UROLOGY', 'TRAMADOL HCL/ACETAMINOPHEN'),
            ( 'UROLOGY', 'TRIMETHOPRIM'),
            ( 'VASCULAR SURGERY', 'ACETAMINOPHEN WITH CODEINE'),
            ( 'VASCULAR SURGERY', 'APIXABAN')],
            names=['specialty', 'medication'], length=2779)
```

Pandas MultiIndex

- When we group by more than one column, the result uses a Multiindex.
 - I.e., the index that involves more than one column.
- Indexing into a Series or DataFrame that has a MultiIndex requires a tuple with at least one value.



```
df = pd.DataFrame([[1, 1, 1], [1, 2, 2], [2, 1, 3], [2, 2, 4]], columns=["A", "B", "C"])
df
```

	A	B	C
0	1	1	1
1	1	2	2
2	2	1	3
3	2	2	4

```
df = df.set_index(["A", "B"])
df.index

MultiIndex([(1, 1),
            (1, 2),
            (2, 1),
            (2, 2)],
            names=['A', 'B'])
```

(1,)

df

		C
	A	B
1	1	1
2	2	2
2	1	3
2	2	4

```
medication_spending_pct[('ADDICTION MEDICINE', 'BUSPIRONE HCL')]
```

```
88.89420255200748
```

```
medication_spending_pct[('ADDICTION MEDICINE', )]
```

```
medication
BUSPIRONE HCL      88.894203
LAMOTRIGINE        8.979849
LORAZEPAM          2.125948
Name: spending_pct, dtype: float64
```

```
medication_spending_pct[('ADDICTION MEDICINE', )].sum()
```

```
100.0
```

```
medication_spending_pct[('ALLERGY/IMMUNOLOGY', )]
```

```
medication
ALBUTEROL SULFATE      3.553199
AMOXICILLIN/POTASSIUM CLAV  0.196089
AZELASTINE HCL         3.646451
AZITHROMYCIN           0.100500
DEXLANSOPRAZOLE       6.561111
DILTIAZEM HCL         0.293666
DOXAZOSIN MESYLATE     0.104872
ENALAPRIL MALEATE      0.318553
FLUTICASONE PROPIONATE  2.842509
FLUTICASONE/SALMETEROL 41.898900
HYDROXYZINE HCL        0.135473
IRBESARTAN            0.058042
LEVOCETIRIZINE DIHYDROCHLORIDE 1.200698
MOMETASONE FUROATE     18.141250
MOMETASONE/FORMOTEROL  2.566123
OMALIZUMAB            15.411051
PREGABALIN            2.679754
RANITIDINE HCL         0.091477
TRIAMCINOLONE ACETONIDE 0.200281
Name: spending_pct, dtype: float64
```

```
medication_spending_pct[('ALLERGY/IMMUNOLOGY', )].sum()
100.0
```

More complex transformations (cont'd.)

- The MultiIndex is sometimes inconvenient to work with, or we may have a better column to index on.
- We can reset (drop) the index using the method `reset_index()`.
 - `reset_index()` converts the MultiIndex into columns and uses a range index (0 to $n - 1$).
 - Allows us to sort on `specialty` and `spending_pct` as we did earlier.

```
spending_df.groupby(["specialty", "medication"])["spending_pct"].sum().head(10)
```

specialty	medication	
ADDICTION MEDICINE	BUSPIRONE HCL	88.894203
	LAMOTRIGINE	8.979849
	LORAZEPAM	2.125948
ALLERGY/IMMUNOLOGY	ALBUTEROL SULFATE	3.553199
	AMOXICILLIN/POTASSIUM CLAV	0.196089
	AZELASTINE HCL	3.646451
	AZITHROMYCIN	0.100500
	DEXLANSOPRAZOLE	6.561111
	DILTIAZEM HCL	0.293666
	DOXAZOSIN MESYLATE	0.104872
Name: spending_pct, dtype: float64		

```
medication_spending_pct = spending_df.groupby(["specialty", "medication"])["spending_pct"].sum().reset_index()
medication_spending_pct.head(10)
```

	specialty	medication	spending_pct
0	ADDICTION MEDICINE	BUSPIRONE HCL	88.894203
1	ADDICTION MEDICINE	LAMOTRIGINE	8.979849
2	ADDICTION MEDICINE	LORAZEPAM	2.125948
3	ALLERGY/IMMUNOLOGY	ALBUTEROL SULFATE	3.553199
4	ALLERGY/IMMUNOLOGY	AMOXICILLIN/POTASSIUM CLAV	0.196089
5	ALLERGY/IMMUNOLOGY	AZELASTINE HCL	3.646451
6	ALLERGY/IMMUNOLOGY	AZITHROMYCIN	0.100500
7	ALLERGY/IMMUNOLOGY	DEXLANSOPRAZOLE	6.561111
8	ALLERGY/IMMUNOLOGY	DILTIAZEM HCL	0.293666
9	ALLERGY/IMMUNOLOGY	DOXAZOSIN MESYLATE	0.104872

```
medication_spending_pct.shape
```

```
(2779, 3)
```

```
medication_spending_pct.sort_values(["specialty", "spending_pct"], ascending=[True, False]).head(10)
```

	specialty	medication	spending_pct
0	ADDICTION MEDICINE	BUSPIRONE HCL	88.894203
1	ADDICTION MEDICINE	LAMOTRIGINE	8.979849
2	ADDICTION MEDICINE	LORAZEPAM	2.125948
12	ALLERGY/IMMUNOLOGY	FLUTICASONE/SALMETEROL	41.898900
16	ALLERGY/IMMUNOLOGY	MOMETASONE FUROATE	18.141250
18	ALLERGY/IMMUNOLOGY	OMALIZUMAB	15.411051
7	ALLERGY/IMMUNOLOGY	DEXLANSOPRAZOLE	6.561111
5	ALLERGY/IMMUNOLOGY	AZELASTINE HCL	3.646451
3	ALLERGY/IMMUNOLOGY	ALBUTEROL SULFATE	3.553199
11	ALLERGY/IMMUNOLOGY	FLUTICASONE PROPIONATE	2.842509

Filtering groups

- Filtering a group is done using the method called `filter()`.
- The method `filter()` takes as input a function name, which it calls on each group of the `DataFrameGroupBy` object.
 - The function must return either `True` or `False`.
 - Groups for which the function returns `False` are dropped.
- The resulting `DataFrame` has its entries in the same order as the original `DataFrame`.

```
df = pd.DataFrame([[1, 1], [1, 2], [2, 3], [2, 4]], columns=["A", "B"])
df
```

	A	B
0	1	1
1	1	2
2	2	3
3	2	4

def my_function(x):
 return x["B"].sum() > 4

```
df.groupby("A").filter(lambda x: x["B"].sum() > 4)
```

	A	B
2	2	3
3	2	4

```
print(len(spending_df['specialty'].unique()))
spending_df['specialty'].unique()[:20]
```

75

```
array(['FAMILY PRACTICE', 'INTERNAL MEDICINE', 'PSYCHIATRY',
      'HEMATOLOGY/ONCOLOGY', 'OPHTHALMOLOGY', 'NEUROLOGY',
      'NURSE PRACTITIONER', 'NEPHROLOGY', 'DENTIST', 'SPECIALIST',
      'GENERAL PRACTICE', 'INTERVENTIONAL CARDIOLOGY',
      'OBSTETRICS/GYNECOLOGY', 'PHYSICIAN ASSISTANT', 'CARDIOLOGY',
      'ENDOCRINOLOGY', 'RHEUMATOLOGY', 'OPTOMETRY',
      'STUDENT IN AN ORGANIZED HEALTH CARE EDUCATION/TRAINING PROGRAM',
      'PULMONARY DISEASE'], dtype=object)
```

```
def filter_on_spending(x):
    return x['spending'].sum() > 50_000

high_spending_df = (spending_df[["specialty", 'spending']]
                    .groupby('specialty')
                    .filter(filter_on_spending))

print(len(high_spending_df['specialty'].unique()))
high_spending_df['specialty'].unique()[:20]
```

38

```
array(['FAMILY PRACTICE', 'INTERNAL MEDICINE', 'PSYCHIATRY',
      'HEMATOLOGY/ONCOLOGY', 'OPHTHALMOLOGY', 'NEUROLOGY',
      'NURSE PRACTITIONER', 'NEPHROLOGY', 'GENERAL PRACTICE',
      'INTERVENTIONAL CARDIOLOGY', 'OBSTETRICS/GYNECOLOGY',
      'PHYSICIAN ASSISTANT', 'CARDIOLOGY', 'ENDOCRINOLOGY',
      'RHEUMATOLOGY', 'OPTOMETRY', 'PULMONARY DISEASE', 'DERMATOLOGY',
      'INTERVENTIONAL PAIN MANAGEMENT', 'PSYCHIATRY & NEUROLOGY'],
      dtype=object)
```

Thinning groups

- Thinning the data consists in reducing the number of entries in a group.
- As opposed to aggregating functions, thinning does not have to reduce the group into a single entry.
 - Although it could reduce it to a single entry.
- Thinning can be used, for instance, to return only the top 3 entries in each category, or to randomly sample a small subset of entries from each category.

Thinning methods and `apply()`

- Pandas offers a few methods for thinning the data.
 - Ex. `nlargest()`, `nsmallest()`
- However, thinning is most often carried out using a method called `apply()`.
- The method `apply()` takes as input a function that it calls on each group of the `DataFrameGroupBy` object.

```
spending_by_specialty['spending'].nsmallest(3)
```

specialty	unique_id	
ADDICTION MEDICINE	TX420809	19.56
	VG585760	82.62
	GJ278932	817.88
ALLERGY/IMMUNOLOGY	HQ120242	109.80
	HN843226	173.05
...		
UROLOGY	VI631849	36.07
	UK512406	79.76
	IE290947	93.30
VASCULAR SURGERY	CU764903	57.59
	KL360002	4309.59

Name: spending, Length: 201, dtype: float64

```
spending_by_specialty['spending'].nlargest(3)
```

specialty	unique_id	
ADDICTION MEDICINE	GJ278932	817.88
	VG585760	82.62
	TX420809	19.56
ALLERGY/IMMUNOLOGY	XY715196	52389.61
	DL492570	29153.71
...		
UROLOGY	XR127617	97962.00
	IW761955	74482.73
	RY513742	28282.76
VASCULAR SURGERY	KL360002	4309.59
	CU764903	57.59

Name: spending, Length: 201, dtype: float64

```
# implementation of nlargest using apply
(spending_by_specialty['spending']
 .apply(lambda x: x.sort_values(ascending=False).head(3))
 )
```

specialty	unique_id	
ADDICTION MEDICINE	GJ278932	817.88
	VG585760	82.62
	TX420809	19.56
ALLERGY/IMMUNOLOGY	XY715196	52389.61
	DL492570	29153.71
		...
UROLOGY	XR127617	97962.00
	IW761955	74482.73
	RY513742	28282.76
VASCULAR SURGERY	KL360002	4309.59
	CU764903	57.59

Name: spending, Length: 201, dtype: float64

Sub-sampling a DataFrame

- Subsampling is the process of selecting a subset of the original data.
- This can be achieved using the DataFrame method called `sample()`.
 - Two parameters are relevant in this scenario, `n` the number of samples to randomly select or `frac` a portion of the data to return.

```
spending_df.sample(frac=0.001)
```

$n = 1,000$

- Subsampling is an ideal way to handle datasets that are unbalanced across some factor(s).

```
# return 1% of the data, i.e., 100 entries
spending_df.sample(frac=0.01)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending	spending_pct
unique_id						
UH807235	1447230941	INTERNAL MEDICINE	ALENDRONATE SODIUM	24	177.18	0.001820
YS953688	1316947484	INTERNAL MEDICINE	CARISOPRODOL	16	663.66	0.006819
DL782194	1598049322	PHYSICIAN ASSISTANT	METOPROLOL TARTRATE	13	79.09	0.006630
NR674778	1407175995	RHEUMATOLOGY	ADALIMUMAB	13	67835.14	7.232658
DK368407	1821079682	FAMILY PRACTICE	METHYLPREDNISOLONE	42	630.56	0.009677
...	...	...	...	...	...	...
PT710704	1023018199	INTERNAL MEDICINE	FINASTERIDE	89	1914.68	0.019672
KZ114129	1740211952	CARDIOLOGY	NADOLOL	12	1707.47	0.089126
HG169995	1417927500	CARDIOLOGY	DIGOXIN	40	926.40	0.048356
TU699302	1205139110	NURSE PRACTITIONER	FELODIPINE	26	738.02	0.022101
GF819597	1508993882	INTERNAL MEDICINE	EZETIMIBE	29	13277.06	0.136415

100 rows × 6 columns

```
spending_df.sample(n=10)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending	spending_pct
unique_id						
OQ100009	1336131606	INTERNAL MEDICINE	TRIMETHOPRIM	13	338.13	0.003474
CR620944	1447305412	NURSE PRACTITIONER	PROCHLORPERAZINE MALEATE	13	173.94	0.005209
AS925249	1801917786	PSYCHIATRY	OLANZAPINE	24	2574.95	0.147274
WF787498	1740549419	EMERGENCY MEDICINE	HYDROCODONE/ACETAMINOPHEN	11	65.72	0.033023
MQ989205	1932204948	FAMILY PRACTICE	CLONAZEPAM	22	236.47	0.003629
KD206211	1811992183	ORTHOPEDIC SURGERY	IBUPROFEN	61	637.72	0.982648
DE165637	1164528105	GENERAL PRACTICE	BECLOMETHASONE DIPROPIONATE	56	8590.64	1.785306
KU856872	1295816684	FAMILY PRACTICE	PHENYTOIN SODIUM EXTENDED	12	551.88	0.008470
RI149272	1104832534	FAMILY PRACTICE	LEVOTHYROXINE SODIUM	67	3788.95	0.058148
LB658428	1588752067	GERIATRIC MEDICINE	ISOSORBIDE DINITRATE	27	1853.74	0.664588

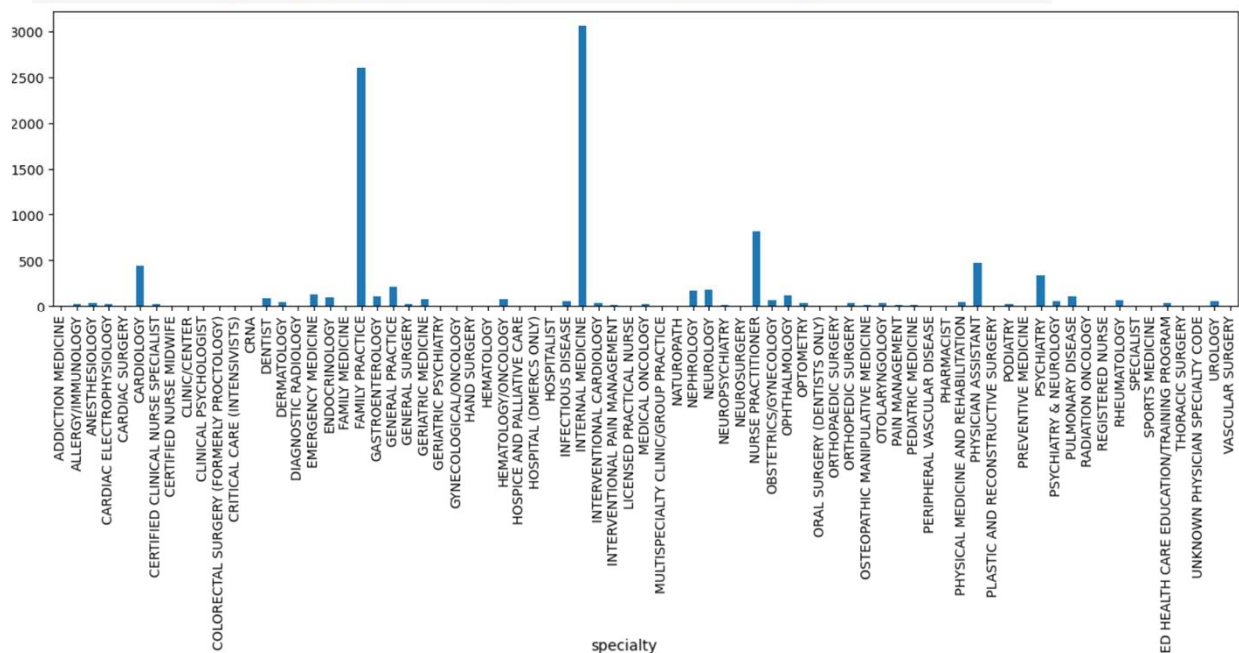
```
spending_df.groupby('specialty').size()
```

```
specialty
ADDICTION MEDICINE                      3
ALLERGY/IMMUNOLOGY                     24
ANESTHESIOLOGY                         30
CARDIAC ELECTROPHYSIOLOGY              20
CARDIAC SURGERY                         2
..
STUDENT IN AN ORGANIZED HEALTH CARE EDUCATION/TRAINING PROGRAM 40
THORACIC SURGERY                       2
UNKNOWN PHYSICIAN SPECIALTY CODE       4
UROLOGY                               59
VASCULAR SURGERY                       2
Length: 75, dtype: int64
```

```
spending_df.groupby('specialty').size().sum()
```

```
10000
```

```
spending_df.groupby('specialty').size().plot.bar(figsize=(16,4))
```



spending_by_specialty.get_group("CARDIOLOGY").sample(n=10)						
	doctor_id	specialty	medication	nb_beneficiaries	spending	spending_pct
unique_id						
SO646228	1336160753	CARDIOLOGY	METFORMIN HCL	18	197.61	0.010315
KB183636	1689679490	CARDIOLOGY	NIFEDIPINE	14	203.50	0.010622
KY789821	1114964442	CARDIOLOGY	OLMESARTAN/HYDROCHLOROTHIAZIDE	41	8938.28	0.466559
LB498598	1023034774	CARDIOLOGY	PRAVASTATIN SODIUM	80	2654.91	0.138581
EK746497	1720034861	CARDIOLOGY	AMIODARONE HCL	75	1583.56	0.082658
TP691165	1871644666	CARDIOLOGY	CLOPIDOGREL BISULFATE	45	594.25	0.031019
GP690572	1932147477	CARDIOLOGY	LISINOPRIL	95	514.78	0.026870
EA476855	1912068297	CARDIOLOGY	HYDROCHLOROTHIAZIDE	27	59.94	0.003129
LC673466	1619181427	CARDIOLOGY	PRAVASTATIN SODIUM	11	591.50	0.030875
AP225670	1124052154	CARDIOLOGY	FLECAINIDE ACETATE	28	1401.28	0.073144

# We sample only 10% of the data in each category						
def sample_10p(x):						
return x.sample(frac=0.1, replace=True)						
spending_by_specialty.apply(sample_10p, include_groups=False).head(10)						
	doctor_id		medication	nb_beneficiaries	spending	spending_pct
specialty	unique_id					
ALLERGY/IMMUNOLOGY	EW891894	1104888403	DEXLANSOPRAZOLE	32	12411.92	6.561111
	LM829997	1063518546	AMOXICILLIN/POTASSIUM CLAV	20	370.95	0.196089
ANESTHESIOLOGY	WD732008	1538235213	OXYCODONE HCL	56	34073.91	23.860491
	XZ425472	1972502284	METAXALONE	15	4604.98	3.224669
	SC540792	1891850012	HYDROMORPHONE HCL	18	565.06	0.395687
CARDIAC ELECTROPHYSIOLOGY	ZA599500	1538130034	FLECAINIDE ACETATE	69	4270.44	1.893399
	XR445715	1811135908	TRAZODONE HCL	19	69.85	0.030970
CARDIOLOGY	RM548848	1043267958	LOSARTAN/HYDROCHLOROTHIAZIDE	11	219.51	0.011458
	NI413073	1912085705	BISOPROLOL FUMARATE	83	4346.15	0.226860
	IC737150	1003009861	BUMETANIDE	17	340.02	0.017748

```
# We select 10 samples of the data in each category
```

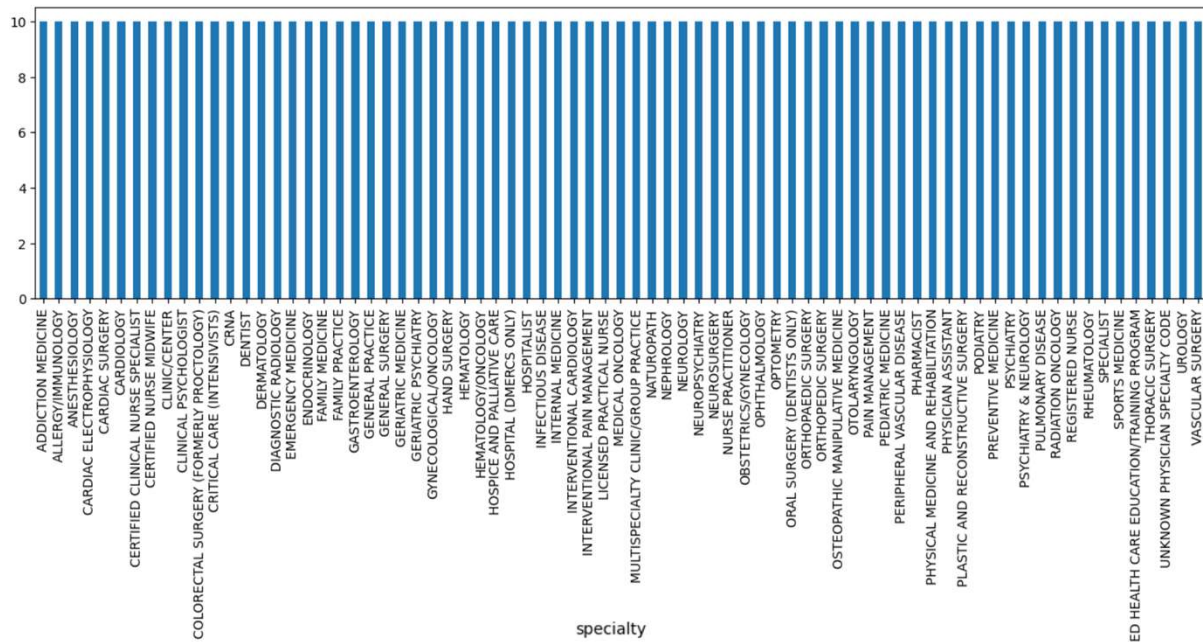
```
def sample_10p(x):
    return x.sample(n=10, replace=True)
```

```
spending_subsampled_df = spending_df.groupby('specialty').apply(sample_10p, include_groups=False)
spending_subsampled_df
```

		doctor_id	medication	nb_beneficiaries	spending	spending_pct
specialty	unique_id					
ADDICTION MEDICINE	VG585760	1801032297	LAMOTRIGINE	11	82.62	8.979849
	GJ278932	1134139991	BUSPIRONE HCL	49	817.88	88.894203
	GJ278932	1134139991	BUSPIRONE HCL	49	817.88	88.894203
	TX420809	1801032297	LORAZEPAM	14	19.56	2.125948
	TX420809	1801032297	LORAZEPAM	14	19.56	2.125948
...	...	...	...	...	...	...
VASCULAR SURGERY	KL360002	1710981444	APIXABAN	14	4309.59	98.681300
	CU764903	1780885574	ACETAMINOPHEN WITH CODEINE	16	57.59	1.318700
	CU764903	1780885574	ACETAMINOPHEN WITH CODEINE	16	57.59	1.318700
	KL360002	1710981444	APIXABAN	14	4309.59	98.681300
	CU764903	1780885574	ACETAMINOPHEN WITH CODEINE	16	57.59	1.318700

750 rows × 5 columns

```
spending_subsampled_df.groupby('specialty').size().plot.bar(figsize=(16,4))
```



Exercise

```
df = pd.read_csv("HI.TXT", header=None,  
                 names=['State', 'Sex', 'Year', 'Name', 'Count'])  
df.head()
```

	State	Sex	Year	Name	Count
0	HI	F	1910	Mary	47
1	HI	F	1910	Helen	20
2	HI	F	1910	Alice	19
3	HI	F	1910	Rose	18
4	HI	F	1910	Violet	16

Dataset source: <https://www.ssa.gov/oact/babynames/state/namesbystate.zip>

- Top 10 most/least popular baby girl/boy names in Hawaii?